To:	Ms. Amenda Raganya (Project Supervisor)
From:	Group 4 – CTU Durban
Module:	Software Engineering 2 (SPM622) – Weekly Submission 3
Date:	16 October 2026
Document:	Software Requirements Specification, version 0.1 (draft)
GitHub:	https://github.com/Taxi-Tap/Week-3-Submission
Project plan:	Linear: https://linear.app/taxi-tap (team TAX)

Taxi Tap: Digital Wallet for Minibus Taxi Fares

Group Members

Name and surname	Student number	Area
Lethukuthula Gumede (Group Lead)	20240280	GitHub, project management, builds, UI design, Google Maps
Londa Mavimbela	20232796	NFC and server actions (wallets, ledger, payments, fares)
Christoffel Du Toit	20240275	QR code, front-end screens, ERD and schema, testing, compiling submissions

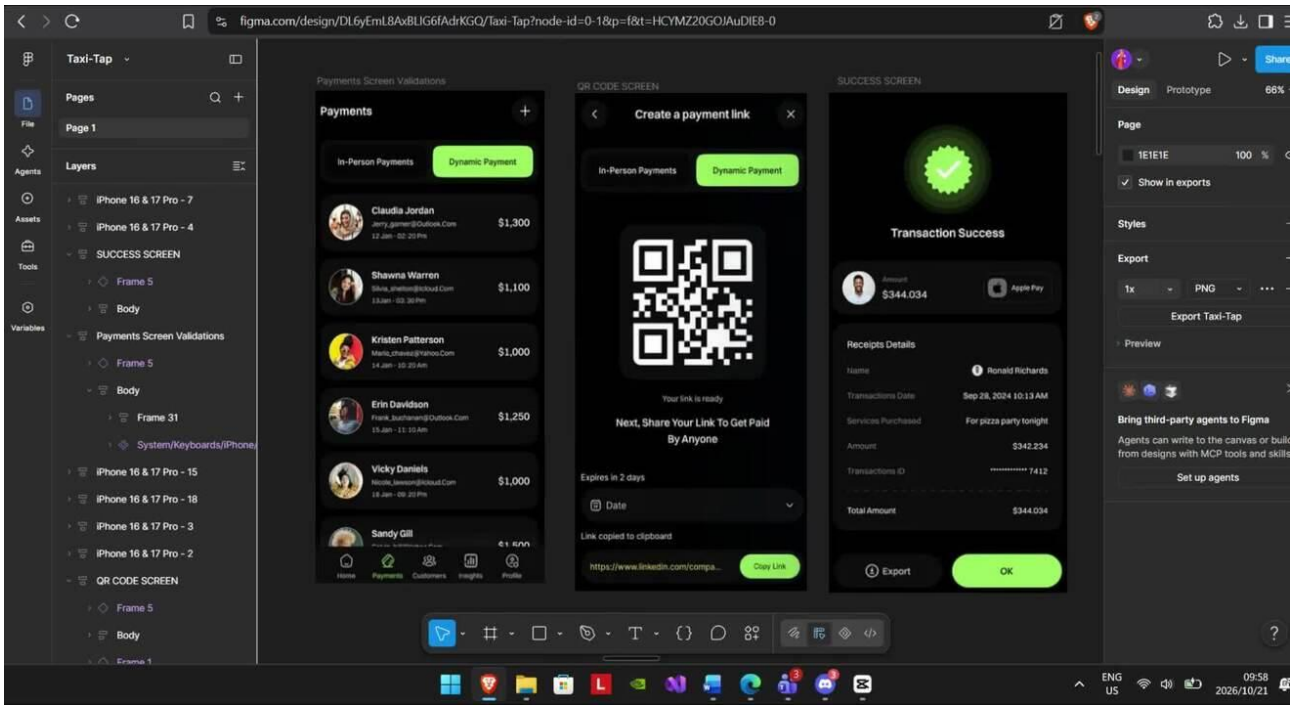
Revision History

Version	Date	Author Durban Group	Description of changes and what was proposed within meetings we had.
0.1	2 October 2026	Group 4	Initial project proposal prepared and Taxi Tap topic submitted for approval.
0.2	5 October 2026	Group 4	Defined the project background, identified the main problem, and established the purpose of the proposed system.
0.3	7 October 2026	Group 4	Developed the initial problem statement, research questions, project aim, and objectives.
0.4	9 October 2026	Group 4	Completed the first drafts of Chapters 1 and 2, including the problem definition and literature review.
0.5	9 October 2026	Group 4	Prepared the initial project plan, assigned team responsibilities, and identified project milestones and risks.
0.6	12 October 2026	Group 4	Reviewed the research findings and refined the project scope, stakeholders, and proposed system features.
0.7	14 October 2026	Group 4	Developed the requirements, user stories, acceptance criteria, and initial functional and non-functional requirements.
0.8	16 October 2026	Group 4	Prepared Chapter 3: Requirements and Methodology, and drafted the Software Requirements Specification (SRS).
0.9	20 October 2026	Group 4	Developed Chapter 4: System Design, including the architecture, UML diagrams, database design, ERD, and interface wireframes.
1.0	21 October 2026	Group 4	Compiled and formatted the assignment, aligned the chapters and requirements, and updated the project progress report and registers.

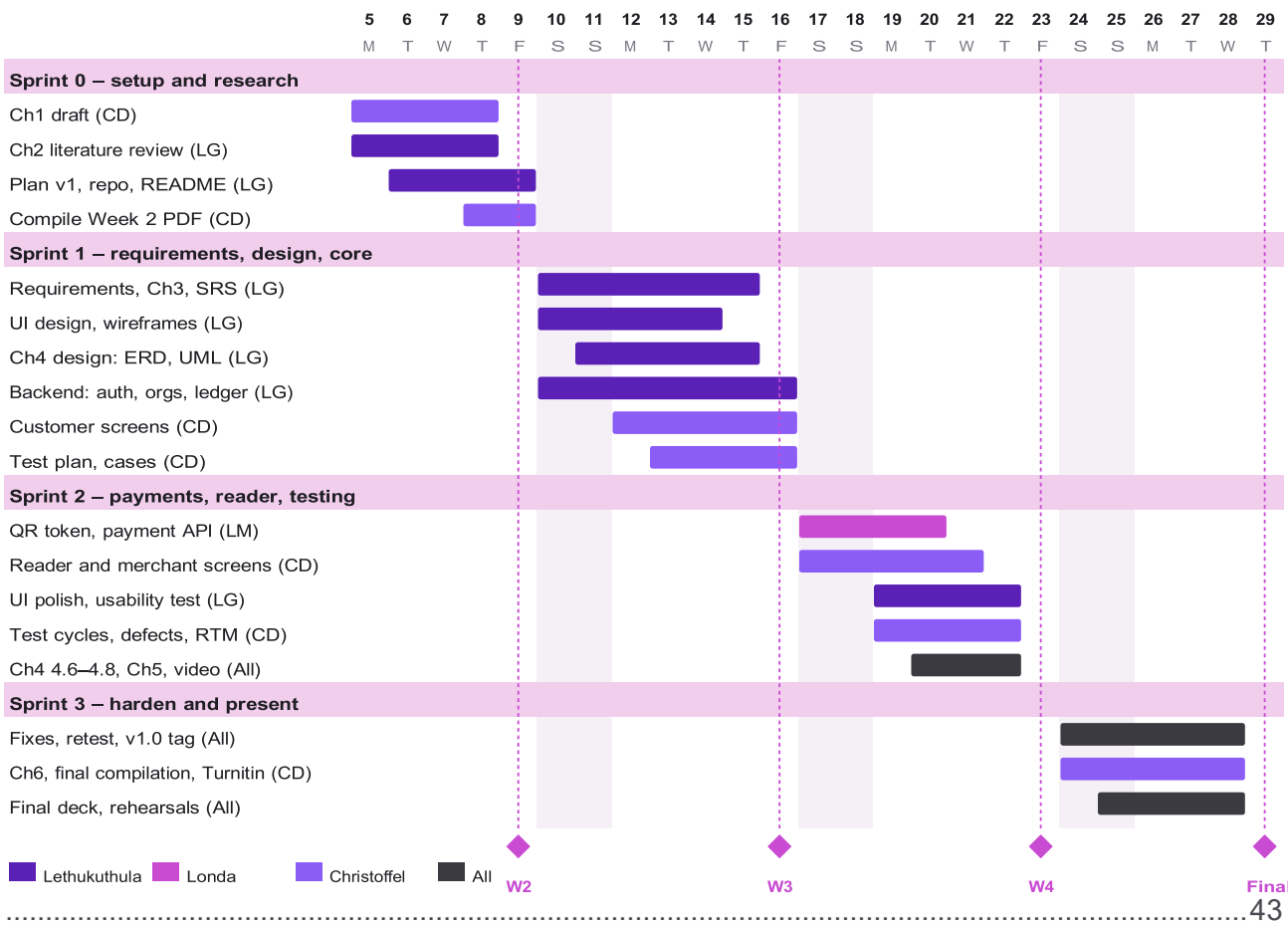
Table of Contents

Group Members 1
Revision History 1
Table of Contents 2
3.1 Introduction 5
3.2 Stakeholders and Users 5
3.3 How Requirements Were Gathered 6
3.4 Functional Requirements 8
3.4.1 Prioritisation 8
3.4.3 Design-only items (not built in this project) 9
3.5 Non-Functional Requirements 10
3.6 User Stories and Acceptance Criteria 11
3.6.1 Traceability to the research gap 12
3.7 Development Methodology 12
3.7.1 Chosen approach: Agile (Scrum-based) 12
3.7.2 From requirement to released code 13
3.7.3 Definition of done 13
3.7.4 Tools that support the method 14
3.7.5 Roles, ownership and review 14
3.7.6 Requirements management and traceability 14
3.7.7 Justification against Waterfall 15
3.8 Ethical, Privacy and Security Considerations 15
3.8.1 Ethical considerations 15
3.8.2 Privacy considerations 16
3.8.3 Security considerations 16
3.9 Summary 16
SOFTWARE REQUIREMENTS SPECIFICATION 17
SRS 1 Introduction 17
1.1 Purpose 17
1.2 Scope 17
1.3 Definitions, Acronyms and Abbreviations 18
1.4 References 18
1.5 Document Overview 18
2. OVERALL DESCRIPTION 18
2.1 Product Perspective 18
2.2 Product Functions 19
2.3 User Classes and Characteristics 19
2.4 Operating Environment 19
2.5 Design and Implementation Constraints 19

2.6 Assumptions and Dependencies 20
3. FUNCTIONAL REQUIREMENTS 21
3.1 Accounts and access 21
3.2 Organisations, routes and fares 21
3.3 Wallet and ledger 21
3.4 Payment methods 22
3.5 Boarding reader 22
3.6 Merchant mode 23
3.7 Owner and association views 23
3.8 Demonstration support 23
4. EXTERNAL INTERFACE REQUIREMENTS 24
4.1 User Interfaces 24
4.2 Hardware Interfaces 24
4.3 Software Interfaces 24
4.4 Communications Interfaces 25
5. NON-FUNCTIONAL REQUIREMENTS 25
6. DATA REQUIREMENTS 26
6.1 Logical Data 26
6.2 Data Rules 26
7. BUSINESS RULES 27
8. USE CASES 27
9. DESIGN-ONLY AND OUT-OF-SCOPE ITEMS 28
10. REQUIREMENTS TRACEABILITY MATRIX 28
11. OPEN ISSUES 29
4.2 System architecture and technology stack 30
Table 1: Technology stack and justification 31
4.3 UML models 32
Use case diagram 32
Sequence diagram 33
Activity diagram 34
4.4 Database design 35
Data integrity and design decisions 36
4.5 User interface design 37



.....37
WEEKLY PROGRESS REPORT AND UPDATED REGISTERS.....39
Weekly Progress Report.....39
Progress by team member.....39
Updated work plan.....39
Milestone status.....40
Risk, Issue and Change Register.....40
Submission completion check.....41
. REFERENCES.....42
Gant Chart.....43



3. REQUIREMENTS AND METHODOLOGY

3.1 Introduction

Chapter 1 defined the problem: cash fares in minibus taxis cause change disputes, theft and poor records.

Chapter 2 showed that earlier cashless projects failed when they ignored drivers' incentives, lacked interoperability or tied commuters to a single purpose card.

This chapter turns those findings into a draft set of requirements for our company Taxi Tap and explains the method the group uses to build it.

It covers the stakeholders and users (3.2), how requirements were gathered (3.3), the functional and non-functional requirements (3.4 and 3.5), the user stories (3.6), the development methodology (3.7), and the ethical, privacy and security considerations (3.8).

The complete requirements list, with acceptance criteria and traceability, is in the Software Requirements Specification (SRS), which this chapter summarises and which the group's testing will trace back to. The system design and the justified technology stack are in Chapter 4.

All requirements are for a working software prototype that uses simulated funds. Items that would need real hardware, bank integration or regulatory approval are recorded as design-only (3.4.3).

3.2 Stakeholders and Users

Chapter 1 identified passengers, operators and drivers, and taxi associations. The literature review showed that taxi owners and informal traders must also be included, because owners need reliable takings data (Schalekamp, McLaren & Behrens 2017) and a wallet that traders can accept with a printed QR is part of the research gap (TechCabal 2025).

Stakeholder	Type	Main interest	Key needs (requirement IDs)
Passengers (commuters)	Primary user	Pay quickly without cash or change; trust that payment worked	FR-02, FR-04, FR-05, FR-11, FR13
Drivers and conductors	Primary user	Fast boarding, no cash counting, no change disputes, a record of the shift	FR-07, FR-08, FR-10, FR-11, FR20
Taxi owners	Primary user	Reliable takings per taxi and driver instead of verbal reports	FR-14, FR-15, FR-22, FR-23
Taxi associations	Administrator and influencer	Routes and fares set correctly; association ownership of the system	FR-06, FR-15
Informal traders (merchants)	Primary user	Accept payment without a card machine	FR-09
Supervisor and CTU	Assessor	Evidence of requirements, design, development and testing	All; NFR-08
Development team (Group 4)	Builder	A buildable scope within four weeks	MoSCoW priorities (3.4.1)
Banks, payment partners and regulators	Future external party	Compliance and safe handling of real money	Design-only (3.4.3)

Table 1: Stakeholders and users

The lesson from BebaPay is that a system fails when one group, here drivers and conductors, loses income or gains no benefit (The Standard 2015; Joseph 2015). Driver needs are therefore first-class requirements and not only commuter needs.

3.3 How Requirements Were Gathered

Because of the four-week timeline and the ethics conditions in Chapter 1, the group combined techniques that do not depend on collecting data from people before consent is confirmed.

1. **Supervisor and topic-acceptance conditions.** The accepted proposal fixed the core scope: customer and merchant modes, a QR payment flow with optional NFC tag reading, a simulated boarding reader with route fare tables, and simulated funds only.
2. **Literature and competitor review (Chapter 2).** Seven existing solutions (Codeta SAPay, Santaco WOW, Waxd, Loop, Tap-I-Fare and FairPay, Street Wallet and BebaPay) were assessed for strengths, limitations and lessons. Each "Lesson for Taxi Tap" and each row of the research gap table in section 2.7 produced one or more requirements.
3. **Stakeholder analysis.** The stakeholders in Table 1 were identified from Chapters 1 and 2, and each was given needs that became requirements.
4. **Scenario analysis.** The group walked through typical journeys: boarding at a rank, boarding at a roadside stop, paying a ladder fare with several destinations, paying for more than one

passenger, a trader requesting payment, and a failed or repeated payment. This is a permitted alternative while taxi association access is pending (risk R02).

5. **User stories and group workshops.** The five user stories in Chapter 1 were expanded and prioritised with MoSCoW in team sessions, and recorded as issues in Linear (team TAX).
6. **Validation (planned).** Requirements will be checked through functional test cases from Sprint 1 and a usability test in Sprint 2. No interviews, surveys or observation of people will take place until ethics confirmation and informed consent are in place (risk R03). [CONFIRM: add any informal feedback or observation the group has actually done, with dates, or delete this sentence if none.]

The requirements in this chapter are a draft baseline. They will be refined in the SRS and at each sprint review as feedback arrives.

3.4 Functional Requirements

3.4.1 Prioritisation

Priorities use the MoSCoW method: **Must** (needed for the working app due on 23 October), **Should** (important, built if time allows) and **Could** (desirable, only after the Must and Should items are done). The group builds Must items first (risk R01). The requirement IDs match those used in the research gap table in section 2.7.

3.4.2 Functional requirements list

ID	Requirement	Priority	Source / link
FR-01	The system shall let a user register and sign in with an email address and password, and keep them signed in with a session until they sign out or the session expires.	Must	Stakeholders (3.2); NFR-03
FR-02	The system shall let a customer view their current wallet balance.	Must	US-01
FR-03	The system shall hold a wallet for each customer, owner business and merchant organisation in the platform's own ledger, with no bank account or card needed.	Must	Schalekamp et al (2017); Loop limits
FR-04	The system shall show a clear outcome (successful, failed or pending) for every payment to both the payer and the receiver, and shall never show a failed or incomplete payment as successful.	Must	Aruho et al (2021) (trust); testing focus
FR-05	The system shall let a customer add demo funds to their wallet.	Must	Simulated funds (Chapter 1); US-02
FR-06	The system shall store routes with fare bands (a label and an amount, using a flat or ladder fare model) that only an association administrator can add or change.	Must	Research gap; US-04
FR-07	The system shall let a driver's phone act as the boarding reader by scanning a customer's QR code with the phone camera, with no dedicated in-vehicle hardware.	Must	Loop; WOW and Codeta hardware costs; US-05
FR-08	The system shall give a driver a shift record that lists each boarding payment and the shift total.	Must	BebaPay; Aruho et al (2021); US06
FR-09	The system shall provide a merchant mode in which a trader enters an amount, shows a payment QR, and a customer pays it from the same wallet.	Should	Street Wallet; Table 2; US-08
FR-10	The system shall confirm a payment to the driver immediately, in the same call as the scan, so the driver does not count cash or give change.	Must	Waxd; BebaPay; US-05
FR-11	The system shall apply the fare of the fare band chosen at boarding, calculated on the server, without requiring a tap-off when the passenger alights.	Must	Research gap; US-04
FR-12	The system shall provide a simulated NFC tap that runs the same payment path as the QR code, for phones without NFC.	Should	Chapter 1 scope; US-03
FR-13	The system shall let a customer view their transaction history with date, amount, route or merchant, and status.	Must	US-10
FR-14	The system shall let an owner view takings per taxi and per driver for a chosen period.	Should	Schalekamp et al (2017); Waxd; US-07
FR-15	The system shall let an association administrator link owner businesses to their association, and link drivers or staff, vehicles and stalls to an owner business.	Must	Stakeholders (associations, owners)

FR-16	The system shall accept a manually typed payment code when QR scanning fails.	Should	Risk R04
FR-17	The system shall refuse a payment when the payer's balance is too low, and shall refuse to process the same payment twice.	Must	Risk R11; NFR-05
FR-18	The system shall include a script that loads and resets demo data (users, taxis, routes, fares and balances).	Should	Risk R09
FR-19	The system shall give each profile one role (customer, driver, owner, association_admin or merchant_staff), and every server action shall check the signed-in user and role before it runs.	Must	Stakeholders (3.2); NFR-03
FR-20	The system shall let a driver start and end a trip by choosing a vehicle and a route, and shall link each fare payment to the open trip.	Must	Boarding at ranks and roadside stops (2.1)
FR-21	The system shall let a driver charge for more than one passenger in a single scan by entering a quantity.	Should	Scenario analysis
ID	**Requirement**	**Priority**	**Source / link**
FR-22	The system shall record the latitude and longitude of each reader payment when the driver allows location access.	Should	Owner accountability; scenario analysis
FR-23	The system shall show an owner their routes and the locations of reader payments on a Google map.	Could	US-07
FR-24	Where the phone supports Web NFC (Android Chrome over HTTPS), the system shall read an NFC tag serial that resolves to a wallet, and shall let tags be linked to wallets.	Could	W3C Web NFC Community Group (n.d.); Chromium Projects (n.d.)
FR-25	The system shall show a customer a QR code for their wallet that carries a signed token which expires after 60 seconds and can be used once.	Must	NFR-03; US-03
FR-26	The system shall move money only through one transfer function that debits the payer, credits the receiver and records the transaction inside a single database transaction.	Must	Risk R11; NFR-05

Table 2: Functional requirements (26: 17 Must, 7 Should, 2 Could)

3.4.3 Design-only items (not built in this project)

In line with the topic acceptance conditions, the following are documented as design only: reader hardware; payment cards and wristbands; a phone-free fallback for commuters; bank linking and PayShap top-ups (BusinessTech 2023); zero-rated or data-light access; an offline driver mode; live fare feeds from taxi associations; and any handling of real money, which would need a licensed payment partner and regulatory assessment.

3.5 Non-Functional Requirements

ID	Category	Requirement and measure	Priority	Verified by
NFR-01	Performance	A scan-to-confirmation payment shall complete within 3 seconds under normal mobile network conditions. The target is set against the reference of about 2.6 seconds for digital payments versus 4.2 seconds for cash (Mondato n.d.).	Must	Timed payments in test cycle 2
NFR-02	Usability	A first-time user shall complete a fare payment without instruction; the usability test targets a System Usability Scale (SUS) score of 68 or higher.	Should	Usability test (TAX-65)
NFR-03	Security	All traffic shall use HTTPS. Passwords shall be stored hashed. Every server action shall check user and role. QR tokens shall be signed, expire after 60 seconds and be single-use, and shall be checked by the server, never the phone. Secrets shall stay out of the repository.	Must	Role-check and token tests; code review of every pull request
NFR-04	Privacy	The system shall collect only the data it needs (name, phone or email, role) and no bank, card or ID data, and shall follow the conditions of the Protection of Personal Information Act (POPIA).	Must	Data review against the schema
NFR-05	Reliability and data integrity	Wallet balances shall always reconcile with the transaction ledger. Amounts shall be stored as whole cents. Ledger writes shall be atomic, and a unique index on the idempotency key shall stop double charges.	Must	Ledger tests (TAX-40); reliability test (TAX-66)
NFR-06	Compatibility	The prototype shall run as an installable web app on current Chrome (Android) and Safari (iOS), and be usable on a 360 px wide screen.	Must	Device checks in Sprint 1 and Sprint 2
NFR-07	Availability	The deployed prototype shall be available during assessment, with a recorded walkthrough as a backup.	Should	Deployment check; walkthrough video (TAX-80)
NFR-08	Maintainability and testability	Code shall be version-controlled with one branch and one reviewed pull request per issue. Lint, type-check and Vitest unit tests shall run on every pull request, and the fare engine and ledger shall have automated unit tests.	Should	CI results; TAX-40, TAX51
ID	**Category**	**Requirement and measure**	**Priority**	**Verified by**
NFR-09	Accessibility	Text and controls shall meet WCAG 2.1 AA contrast and size guidance so they stay readable on small screens and in sunlight.	Should	Design review of the design system (TAX85)
NFR-10	Data use	Payment screens shall be lightweight to suit users with limited data.	Could	Page size check

Table 3: Non-functional requirements

3.6 User Stories and Acceptance Criteria

The five user stories in Chapter 1 are extended below to cover drivers, owners, traders and associations. Each story has acceptance criteria that become test cases in the test plan. The UML use case diagram is presented in Chapter 4.

ID	User story	Acceptance criteria	Links
US-01	As a passenger, I want to view my wallet balance so that I know how much I can spend on rides.	Balance shown after sign-in; matches the ledger; updates after every payment.	FR-02, FR-03
US-02	As a passenger, I want to add demo funds so that I can pay for rides.	Balance rises by the exact amount; the top-up appears in history.	FR-05, FR-13, FR-26
US-03	As a passenger, I want to pay by showing a QR code (or a simulated NFC tap) so that I can pay without cash.	Payment succeeds when funds are enough; clear success or failure message; balance reduced once only.	FR-04, FR-07, FR-12, FR-17, FR-25
US-04	As a passenger, I want the correct fare for my route and destination to be applied when I board so that I never argue about change.	Fare equals the fare band amount for the route, times the number of passengers; no tap-off needed.	FR-06, FR-11, FR-21
US-05	As a driver, I want to scan a customer's code and see instant confirmation so that I never handle cash.	Confirmation within 3 seconds; failed payments are clearly rejected.	FR-07, FR-10, FR-20, NFR-01
US-06	As a driver, I want a shift record so that I can show what I collected.	Shift list and total match the ledger for that driver.	FR-08, FR-20
US-07	As an owner, I want takings per taxi and driver, and a map of where payments happened, so that I no longer rely on verbal reports.	Totals per taxi and driver match the ledger; payment pins appear on the route map.	FR-14, FR-22, FR-23
US-08	As a trader, I want to request an amount and get paid without a card machine so that customers can pay digitally.	Customer scans the trader's payment QR and pays the requested amount; trader sees confirmation.	FR-09, FR-04
US-09	As an association administrator, I want to link owners, drivers, vehicles and stalls to my association and maintain route fares so that payments land in the right place.	Linked drivers' payments appear under the correct owner and taxi; fare edits apply to later boardings only.	FR-01, FR-06, FR-15, FR-19
US-10	As a passenger, I want to see my transaction history so that I can track payments and their statuses.	History lists date, amount, route or merchant, and status in order.	FR-13

Table 4: User stories with acceptance criteria

3.6.1 Traceability to the research gap

Table 5 links each gap from section 2.7 to the requirements above, so the group can show that every requirement has a reason and every gap is covered.

Gap identified	Evidence	Requirements
Fares must be set at boarding without tap-off	Schalekamp et al (2017); taxi review	FR-06, FR-07, FR-11
No dedicated in-vehicle hardware	WOW and Codeta hardware costs; Loop	FR-07
Gap identified	**Evidence**	**Requirements**
Must not depend on a bank account	Schalekamp et al (2017); Loop limits	FR-03
Drivers need a benefit	BebaPay; Aruho et al (2021)	FR-08, FR-10
Owners need reliable takings data	Schalekamp et al (2017); Waxd	FR-14
One wallet across taxis and traders	Street Wallet; Table 2	FR-09
Trust and reliability	Aruho et al (2021)	FR-04, NFR-05

Table 5: Research gap to requirements

3.7 Development Methodology

3.7.1 Chosen approach: Agile (Scrum-based)

The group uses an Agile software development life cycle (SDLC) based on Scrum, with one-week sprints that finish on each weekly submission (Beck et al 2001; Schwaber & Sutherland 2020). All planning is done in Linear (team TAX), where every piece of work is an issue linked to a requirement. Table 6 shows the sprint plan.

Sprint	Dates (2026)	Focus	Output
Sprint 0	5–9 October	Setup and research	Chapters 1–2, plan v1, repository
Sprint 1	9–16 October	Requirements, design and core backend	Chapter 3, SRS, Chapter 4 design, schema script; release v0.1 (16 Oct)
Sprint 2	16–23 October	Payments, reader and testing	Working app (Must requirements), test cycle 2; release v0.2 (23 Oct)
Sprint 3	23–29 October	Harden and present	Fixes, final compilation, demo; release v1.0 (28 Oct)

Table 6: Sprint plan

3.7.2 From requirement to released code

Every piece of code starts as a Linear issue and ends as a reviewed pull request that closes it. This gives the commit, branch and review evidence that the module assesses.

1. **Plan.** A requirement from this chapter becomes one or more Linear issues, tagged with its MoSCoW priority. Must issues are scheduled first.
2. **Branch.** The developer moves the issue to In Progress and creates a feature branch using the name Linear suggests, for example `tax-41-signed-short-lived-qr-token`.
3. **Commit.** Commits are small and frequent, and each message says what changed and names the issue, for example "Add 60-second expiry to QR tokens (TAX-41)".
4. **Pull request.** The pull request description contains `Fixes TAX-41`, so Linear links it and closes the issue when it merges.
5. **Review and checks.** The reviewer for that folder approves the pull request, and the continuous integration (CI) checks must pass: lint, type-check and Vitest unit tests.
6. **Merge.** The pull request is squash-merged into the protected main branch (no direct pushes, at least one approval), the branch is deleted, and the Vercel preview is checked on a phone.
7. **Review the sprint.** A short daily stand-up and a sprint review at each weekly boundary reprioritise the backlog, and a release tag marks each working increment.
8.

3.7.3 Definition of done

- The acceptance criteria of the linked requirement are met.
- The pull request is approved by the folder reviewer and CI is green.
- Server logic that handles money (ledger, fare engine) has unit tests.
- The feature works on the phone preview on Android Chrome and, for camera features, iOS Safari.
- The Linear issue is closed by the merge, and evidence is filed in the `capstone/` folder where needed.

3.7.4 Tools that support the method

Practice	Tool	Purpose
Planning and backlog	Linear (team TAX)	Issues, sprints and owners, linked to GitHub branches and pull requests
Version control	GitHub (one repository, taxi-tap)	Branch and pull request per issue; protected main branch
Continuous integration	GitHub Actions	Lint, type-check and unit tests on every pull request
Preview and hosting	Vercel (proposed; to be confirmed)	A preview link for each pull request over HTTPS, needed to test camera and NFC on phones
Database environments	Neon branch for each developer	Each developer works on their own copy of the data, so changes do not clash
Same setup on every laptop	npm run setup, doctor and check	Installs, checks and prepares the project the same way for everyone
Testing	Vitest and test workbooks	Automated unit tests and recorded test cycles with a requirements traceability matrix

Table 7: Tools that support the method

3.7.5 Roles, ownership and review

Each folder of the code has one owner, and a different member reviews every pull request that touches it. Every member commits code in every sprint, including documentation-only weeks.

Member	Area	Owns	Reviews
Lethukuthula Gumede	GitHub, project management, builds, UI design, Google Maps	Owner screens and maps, UI components and theme, authentication setup, setup scripts, CI, capstone evidence	Customer, merchant and authentication screens; server actions
Londa Mavimbela	NFC and server actions	NFC components, server actions (wallets, ledger, payments, fares, organisations), seed script	Reader and QR code; database schema; authentication setup, CI
Christoffel Du Toit	QR code, front-end screens, ERD and schema, testing, compiling submissions	Authentication, customer, merchant and reader screens; QR components; database schema and migrations	NFC; owner screens and maps; UI components

Table 8: Roles, ownership and review

3.7.6 Requirements management and traceability

Each requirement has an ID and a MoSCoW priority, and is traced in the SRS to its source, its Linear issues and its test case. The test workbooks hold the requirements traceability matrix. A change to a Must requirement is logged in the Risk, Issue and Change Register and agreed at stand-up before the backlog is changed.

3.7.7 Justification against Waterfall

The plan-driven alternative considered was the Waterfall model, in which requirements, design, implementation, testing and release follow one another as separate phases (Royce 1970).

Criterion	Agile (Scrum-based)	Waterfall	Fit for Taxi Tap
Fixed four-week timeline	Delivers a working increment every week	Working software appears only after the build phase	Agile
Changing requirements	Backlog reprioritised each sprint	Changes after sign-off are costly	Agile (association access is delayed, risk R02)
Feedback	Supervisor feedback weekly, applied in the next sprint	Feedback mainly at the end	Agile
Early testing	Tests and test cycles from Sprint 1	Testing late in the cycle	Agile (risk R07)
Documentation	Lighter, so it must be planned	Heavy upfront documentation	Mixed: the module needs a formal SRS and design
Planning predictability	Needs discipline in a small team	Clear phases and milestones	Waterfall

Table 9: Agile compared with Waterfall for Taxi Tap

Agile was chosen because the timeline is fixed, requirements are still being validated, and the module asks for coding to run alongside documentation from Week 3. The weakness of Agile for a student project is loose documentation, so the group keeps a lightweight upfront baseline (this chapter, the SRS and the Chapter 4 design) and updates it after each sprint review. Fixing everything before coding, as Waterfall requires, would have delayed a working application and left no time to correct problems found in testing.

3.8 Ethical, Privacy and Security Considerations

3.8.1 Ethical considerations

- **Ethics approval and consent.** No interviews, surveys or tests with people will take place until ethics confirmation is received. Participants will receive an information sheet, consent voluntarily and can withdraw at any time. Responses will be anonymous.
- **No real money.** All balances are simulated, so no participant can lose real money by testing the prototype. The prototype does not claim to be a regulated payment service.
- **Fairness to all stakeholders.** The design follows the lesson from BebaPay that a system must not remove drivers' income without a benefit (Joseph 2015), and avoids locking commuters into a single-purpose card (Schalekamp, McLaren & Behrens 2017).
- **Inclusion.** The prototype assumes a smartphone, which excludes some commuters. The phone-free fallback is documented as future work.
- **Honest reporting.** Limitations, including simulated funds and dummy fare data, are stated openly. Any use of AI tools is declared to the supervisor in line with the CTU Referencing and Style Guide (2025).

3.8.2 Privacy considerations

A real version of Taxi Tap would handle personal information, so the prototype follows the conditions of the Protection of Personal Information Act 4 of 2013 (POPIA) as good practice (Republic of South Africa 2013):

- **Data minimisation:** only name, contact detail and role are collected; no ID numbers, bank details or card numbers.
- **Purpose limitation:** data is used only to run and test the prototype.
- **Security safeguards:** access is limited by role, and secrets are kept out of the repository.
- **Retention:** test accounts and any research data are deleted after assessment, unless the supervisor requires otherwise.
- **Demo data:** seeded accounts use fictional names and dummy fares, and each developer works on a separate database branch.
-

3.8.3 Security considerations

Threat	Control	Requirement
Unauthorised access to accounts	Sign-in with hashed passwords and sessions; role checked first in every server action	FR-01, FR-19, NFR03
Interception of data	HTTPS for all traffic (also needed for the camera and Web NFC)	NFR-03
Copied or replayed QR codes	Signed token that expires after 60 seconds and works once; the server checks it, not the phone	FR-25, NFR-03
Double charge or wrong balance	One transfer function in a single database transaction; unique idempotency key on each payment; reconciliation test	FR-17, FR-26, NFR05
Fraud by changing fares or amounts	Only association administrators edit fare bands; the fare is calculated on the server, not on the phone	FR-06, FR-11
Leaked keys or secrets	No keys in the repository; secrets live in Vercel settings and local environment files	NFR-03
Loss of evidence of a payment	Every payment is recorded as a transaction with its trip, driver and fare band	FR-04, FR-13

Table 10: Main security threats and controls

3.9 Summary

This chapter set out a draft baseline of 26 functional and 10 non-functional requirements, prioritised with MoSCoW and traced to the research gap from Chapter 2. It also defined the stakeholders, how requirements were gathered, user stories with acceptance criteria, and the ethical, privacy and security considerations. An Agile (Scrum-based) life cycle, run through Linear, GitHub pull requests and continuous integration, was chosen over Waterfall because of the fixed timeline, evolving requirements and the need for early testing. The requirements are completed in the SRS and are the basis for the system design in Chapter 4.

SOFTWARE REQUIREMENTS SPECIFICATION

SRS 1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) states what Taxi Tap must do. It is the complete requirements list that Chapter 3 summarises, and it is the document that the group's test cases trace back to. It is written for the development team, the project supervisor and anyone who marks or tests the prototype.

1.2 Scope

Taxi Tap is a digital wallet for minibus taxi fares. Passengers hold a wallet and pay with a QR code or a simulated NFC tap. A driver's phone acts as the boarding reader, which applies the fare for the route chosen at boarding. Traders can accept payment in a merchant mode with a payment QR. Owners and associations see takings, routes and fares.

In scope: customer, driver, merchant, owner and association administrator screens; wallets and a ledger; QR payments; simulated NFC tap and optional Web NFC tag reading on Android; route fare bands; trips; takings views; demo data. All money is simulated.

Design-only (not built): see section 9.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
Wallet	A balance held in the platform's ledger for a customer, owner business or merchant organisation.
Ledger	The set of transactions that records every movement of money; balances must always reconcile with it.
Fare band	A named fare on a route (for example "to Berea") with an amount in cents.
Ladder fare	A route with several fare bands, depending on where the passenger gets off.
Trip	A period in which a driver operates one vehicle on one route; fare payments are linked to it.
Boarding reader	The driver's phone running the reader screens, which scans the customer's code.
Merchant mode	The screens that let a trader request an amount and be paid from a customer wallet.
QR token	A signed value inside a QR code that identifies a wallet; it expires after 60 seconds and works once.
Idempotency key	A unique value sent with each payment so the same payment cannot be processed twice.
MoSCoW	Must, Should, Could: the priority scale used for requirements.
PWA	Progressive web app: a website that can be installed on a phone's home screen.
NFC / Web NFC	Near field communication; Web NFC is the browser interface (NDEFReader) that reads tags, on Android Chrome only.
POPIA	Protection of Personal Information Act 4 of 2013.
SUS	System Usability Scale, a standard usability questionnaire.

Table 1: Definitions and abbreviations

1.4 References

Chapters 1 and 2 of the Group 4 dissertation (Weekly Submission 2); Chapter 3: Requirements and Methodology (draft, Weekly Submission 3); Group 4 Tech Stack and Project Structure (15 October 2026); and the sources listed in section 12.

1.5 Document Overview

Section 2 describes the product and its users. Section 3 lists the functional requirements by feature.

Sections 4 to 7 give interfaces, non-functional requirements, data requirements and business rules.

Section 8 gives us use cases. Section 9 lists what is not built. Section 10 is the traceability matrix, and section 11 lists then it opens issues.

2. OVERALL DESCRIPTION

2.1 Product Perspective

Taxi Tap is a new, self-contained web application. It is one Next.js app that runs in the phone browser over HTTPS and can be installed to the home screen.

Screens call server actions in the same app, and those actions read and write a Neon Postgres database through Drizzle ORM.

BetterAuth runs inside the app and stores users and sessions in the same database. The only outside services are Google Maps (route maps) and the hosting platform.

There is no connection to any bank or payment provider.

2.2 Product Functions

- **Customers** register, sign in, add demo funds, view a balance and history, and pay by QR code or simulated NFC tap.
- **Drivers** start a trip, scan customers' codes at boarding, see instant confirmation and view a shift record.
- **Traders** request an amount and receive payment into a merchant wallet.
- **Owners** see takings per taxi and driver, and payment locations on a route map.
- **Association administrators** link owners, drivers, vehicles and stalls, and maintain routes and fare bands.

2.3 User Classes and Characteristics

User class (role)	Description	Access	Main requirements
Passenger (customer)	Commuter with a smartphone; may be a first-time user	Own wallet, payments and history	FR-02, FR-05, FR-13, FR-25
Driver or conductor (driver)	Collects fares on the road; needs speed and simplicity	Reader screens, own trips and shift record	FR-07, FR-08, FR-10, FR-20
Trader (merchant_staff)	Informal trader or stall holder with only a phone	Merchant screens for own stall	FR-09
Owner (owner)	Owns one or more taxis; wants reliable takings data	Own taxis, drivers, takings and map	FR-14, FR-22, FR-23
Association administrator (association_admin)	Manages members, routes and fares for an association	Own association's organisations, routes and fares	FR-06, FR-15

Table 2: User classes

2.4 Operating Environment

- Current Chrome on Android and current Safari on iOS, on phones about 360 px wide or larger; a desktop browser for owner and administrator screens.
- A phone camera for scanning QR codes, and a data connection. HTTPS is required for the camera, location and Web NFC.
- Web NFC tag reading works on Android Chrome only; other phones use the simulated tap.

2.5 Design and Implementation Constraints

Constraint	Detail
Simulated funds	No real money; no bank or payment provider (topic acceptance condition). A licensed partner would hold real money in production.
Timeline	Four weeks: Must requirements working by 23 October; final submission 29 October.
Technology stack	Next.js (React, TypeScript, App Router), shadcn/ui with Tailwind CSS, server actions, Neon Postgres, Drizzle ORM, BetterAuth, Google Maps JavaScript API. Hosting on Vercel is proposed and still to be confirmed. The justified stack is in Chapter 4.
Database access	Only server code imports the database client; screens call server actions only. The database changes only through the schema file and generated migrations. The Drizzle neon-serverless driver is used because the ledger needs database transactions.
Money handling	Amounts are stored as whole cents, and money moves only through one transfer function.
Secrets	No keys or secrets in the repository; they live in hosting settings and local environment files.
Web NFC	Android Chrome over HTTPS only.
Development process	Agile with one issue, one branch and one pull request; main is protected; CI runs lint, type-check and Vitest (see Chapter 3, section 3.7).

Table 3: Design and implementation constraints

2.6 Assumptions and Dependencies

- Users have a smartphone with a camera and some data; a phone-free option is design-only.
- Fare data is dummy data created by the group, because live association fares are not available (risk R02).
- Taxi association access and ethics confirmation may be delayed; no research data from people is collected until consent is confirmed (risk R03).
- A Google Cloud billing account is needed for Google Maps, and the QR library must be chosen by 17 October (section 11).

3. FUNCTIONAL REQUIREMENTS

Each requirement has an ID, a MoSCoW priority, an acceptance criterion that becomes a test case, and a source. "Source" shows where the need comes from (user story, research or risk). The last column shows Linear issues (team TAX) that build it. The IDs match Chapter 3 and the research gap table in section 2.7 of Chapter 2. There are 26 functional requirements: 17 Must, 7 Should and 2 Could.

3.1 Accounts and access

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-01	The system shall let a user register and sign in with an email address and password, and keep them signed in with a session until they sign out or the session expires.	Must	Valid details create an account and session; a wrong password is rejected with a clear message; signed-out users cannot open protected screens.	Stakeholders (3.2); NFR-03 TAX-31, TAX-32
FR-19	The system shall give each profile one role (customer, driver, owner, association_admin or merchant_staff), and every server action shall check the signed-in user and role before it runs.	Must	A customer who calls a reader or owner action is refused; each role sees only its own screens.	Stakeholders (3.2); NFR-03 TAX-33

Table 4: Functional requirements: accounts and access

3.2 Organisations, routes and fares

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-15	The system shall let an association administrator link owner businesses to their association, and link drivers or staff, vehicles and stalls to an owner business.	Must	A driver's fare payments are credited to the wallet of the driver's owner business; each vehicle belongs to one owner and one route.	Stakeholders (associations, owners) TAX-34, TAX-35
FR-06	The system shall store routes with fare bands (a label and an amount, using a flat or ladder fare model) that only an association administrator can add or change.	Must	Each route lists its fare bands in order; a fare change applies only to later payments and never alters recorded transactions.	Research gap; US-04 TAX-47

Table 5: Functional requirements: organisations, routes and fares

3.3 Wallet and ledger

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-03	The system shall hold a wallet for each customer, owner business and merchant organisation in the platform's own ledger, with no bank account or card needed.	Must	A new customer can hold and spend demo funds without entering any bank details.	Schalekamp et al (2017); Loop limits TAX-36

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-02	The system shall let a customer view their current wallet balance.	Must	The balance is shown after sign-in, equals the ledger total for that wallet, and updates after every payment.	US-01 TAX-36
FR-05	The system shall let a customer add demo funds to their wallet.	Must	The balance rises by the exact amount and the top-up is recorded as a transaction of kind "topup".	Simulated funds (Chapter 1); US-02 TAX-37
FR-26	The system shall move money only through one transfer function that debits the payer, credits the receiver and records the transaction inside a single database transaction.	Must	If any step fails, nothing changes; wallet balances always reconcile with the transaction ledger.	Risk R11; NFR-05 TAX-38, TAX-40
FR-17	The system shall refuse a payment when the payer's balance is too low, and shall refuse to process the same payment twice.	Must	A payment above the balance is rejected with no balance change; repeating a payment with the same idempotency key returns the first result and charges once.	Risk R11; NFR-05 TAX-38, TAX-44
FR-13	The system shall let a customer view their transaction history with date, amount, route or merchant, and status.	Must	History lists the customer's transactions newest first, and matches the ledger.	US-10 TAX-39

Table 6: Functional requirements: wallet and ledger

3.4 Payment methods

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-25	The system shall show a customer a QR code for their wallet that carries a signed token which expires after 60 seconds and can be used once.	Must	An expired or reused token is rejected by the server; the screen refreshes the code automatically.	NFR-03; US-03 TAX-41, TAX-42
FR-12	The system shall provide a simulated NFC tap that runs the same payment path as the QR code, for phones without NFC.	Should	A simulated tap produces the same ledger result as a QR payment and is recorded with method "simulated".	Chapter 1 scope; US-03 TAX-46
FR-24	Where the phone supports Web NFC (Android Chrome over HTTPS), the system shall read an NFC tag serial that resolves to a wallet, and shall let tags be linked to wallets.	Could	A linked tag read on a supported phone starts a payment for that wallet; unsupported phones show the simulate-tap button instead.	W3C Web NFC Community Group (n.d.); Chromium Projects (n.d.) TAX-46, TAX-91
FR-16	The system shall accept a manually typed payment code when QR scanning fails.	Should	A valid code completes the same payment as a scan.	Risk R04 TAX-43

Table 7: Functional requirements: payment methods

3.5 Boarding reader

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-20	The system shall let a driver start and end a trip by choosing a vehicle and a route, and shall link each fare payment to the open trip.	Must	A fare cannot be taken without an open trip; a trip shows all its payments.	Boarding at ranks and roadside stops (2.1) TAX-48, TAX-50, TAX-52
FR-07	The system shall let a driver's phone act as the boarding reader by scanning a customer's QR code with the phone camera, with no dedicated in-vehicle hardware.	Must	Scanning reads the token on current Chrome (Android) and Safari (iOS).	Loop; WOW and Codeta hardware costs; US-05 TAX-43, TAX-48
FR-11	The system shall apply the fare of the fare band chosen at boarding, calculated on the server, without requiring a tap-off when the passenger alights.	Must	The amount charged equals the fare band amount times the quantity; the phone cannot change the amount.	Research gap; US-04 TAX-44, TAX-49, TAX-51

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-21	The system shall let a driver charge for more than one passenger in a single scan by entering a quantity.	Should	The charge equals the fare times the quantity, and the quantity is stored with the transaction.	Scenario analysis TAX-49
FR-10	The system shall confirm a payment to the driver immediately, in the same call as the scan, so the driver does not count cash or give change.	Must	The reader shows success or failure, the amount and a passenger reference within 3 seconds.	Waxd; BebaPay; US-05 TAX-44, TAX-52
FR-04	The system shall show a clear outcome (successful, failed or pending) for every payment to both the payer and the receiver, and shall never show a failed or incomplete payment as successful.	Must	A forced failure (low balance or expired token) shows "failed" on both screens and changes no balance.	Aruho et al (2021) (trust); testing focus TAX-44, TAX-50
FR-08	The system shall give a driver a shift record that lists each boarding payment and the shift total.	Must	The shift total equals the ledger total for that driver's trips.	BebaPay; Aruho et al (2021); US-06 TAX-52
FR-22	The system shall record the latitude and longitude of each reader payment when the driver allows location access.	Should	The transaction stores the location when permission is given; the payment still succeeds if permission is refused.	Owner accountability; scenario analysis TAX-53

Table 8: Functional requirements: boarding reader

3.6 Merchant mode

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-09	The system shall provide a merchant mode in which a trader enters an amount, shows a payment QR, and a customer pays it from the same wallet.	Should	The customer sees the merchant name and amount before confirming; both sides see the outcome.	Street Wallet; Table 2; US-08 TAX-55, TAX-56

Table 9: Functional requirements: merchant mode

3.7 Owner and association views

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-14	The system shall let an owner view takings per taxi and per driver for a chosen period.	Should	Totals per taxi and driver match the ledger for the period.	Schalekamp et al (2017); Waxd; US-07 To be linked in Linear
FR-23	The system shall show an owner their routes and the locations of reader payments on a Google map.	Could	Payment pins appear at the stored locations on the route map.	US-07 TAX-90

Table 10: Functional requirements: owner and association views

3.8 Demonstration support

ID	Requirement	Priority	Acceptance criterion	Source; Linear issues
FR-18	The system shall include a script that loads and resets demo data (users, taxis, routes, fares and balances).	Should	Running the script gives a known starting state that can be repeated.	Risk R09 TAX-29, TAX-93

Table 11: Functional requirements: demonstration support

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interfaces

The interface is mobile first, installable as a web app, and themed in the Taxi Tap purple style using shadcn/ui components. The screens are grouped by role as follows.

Screen group	Screens	Role	Purpose
Authentication	Sign-in, sign-up	All	Register and sign in (FR-01)
Customer	Wallet, pay, history	customer	Balance, top-up, QR code, transaction history (FR-02, FR-05, FR-13, FR-25)
Boarding reader	Trip, scan	driver	Start and end a trip, pick the fare band and quantity, scan, see confirmation and shift record (FR-07, FR-08, FR-10, FR-20)
Merchant	Request, scan	merchant_staff	Request an amount and show the payment QR (FR-09)
Owner	Dashboard, map	owner	Takings per taxi and driver, route map (FR-14, FR-23)
Administration	Organisations, routes and fares	association_admin	Link members and maintain fare bands (FR-06, FR-15)

Table 12: User interface groups

Every screen shall show clear success and failure messages, large touch targets, and readable contrast (NFR-02, NFR-09). The wireframes are in Chapter 4.

4.2 Hardware Interfaces

- The phone camera, used to scan QR codes.
- The phone's NFC reader, used only for optional tag reading on Android. No external readers, cards or wristbands are built.
- The phone's location services, used only to stamp reader payments when the driver allows it (FR-22).
-

4.3 Software Interfaces

Interface	Use
BetterAuth (Drizzle adapter)	Email and password sign-in, sessions and roles, stored in the Neon database.

Interface	Use
Neon Postgres via Drizzle ORM	All data, through the neon-serverless driver so that database transactions work.
Google Maps JavaScript API	Route maps and payment pins on the owner screens; key restricted to the project's domains.
QR scanner and generator library	To be chosen by 17 October; must scan on Android Chrome and iOS Safari.
Web NFC (NDEFReader)	Optional tag reading on Android Chrome over HTTPS.

Table 13: Software interfaces

4.4 Communications Interfaces

All communication between the phone and the app uses HTTPS. The app talks to the database over a secure connection. QR tokens are checked by the server and never by the phone.

5. NON-FUNCTIONAL REQUIREMENTS

ID	Category	Requirement and measure	Priority	Verified by
NFR-01	Performance	A scan-to-confirmation payment shall complete within 3 seconds under normal mobile network conditions. The target is set against the reference of about 2.6 seconds for digital payments versus 4.2 seconds for cash (Mondato n.d.).	Must	Timed payments in test cycle 2
NFR-02	Usability	A first-time user shall complete a fare payment without instruction; the usability test targets a System Usability Scale (SUS) score of 68 or higher.	Should	Usability test (TAX-65)
NFR-03	Security	All traffic shall use HTTPS. Passwords shall be stored hashed. Every server action shall check user and role. QR tokens shall be signed, expire after 60 seconds and be single-use, and shall be checked by the server, never the phone. Secrets shall stay out of the repository.	Must	Role-check and token tests; code review of every pull request
NFR-04	Privacy	The system shall collect only the data it needs (name, phone or email, role) and no bank, card or ID data, and shall follow the conditions of the Protection of Personal Information Act (POPIA).	Must	Data review against the schema
NFR-05	Reliability and data integrity	Wallet balances shall always reconcile with the transaction ledger. Amounts shall be stored as whole cents. Ledger writes shall be atomic, and a unique index on the idempotency key shall stop double charges.	Must	Ledger tests (TAX-40); reliability test (TAX-66)
NFR-06	Compatibility	The prototype shall run as an installable web app on current Chrome (Android) and Safari (iOS), and be usable on a 360 px wide screen.	Must	Device checks in Sprint 1 and Sprint 2
NFR-07	Availability	The deployed prototype shall be available during assessment, with a recorded walkthrough as a backup.	Should	Deployment check; walkthrough video (TAX-80)
NFR-08	Maintainability and testability	Code shall be version-controlled with one branch and one reviewed pull request per issue. Lint, type-check and Vitest unit tests shall run on every pull request, and the fare engine and ledger shall have automated unit tests.	Should	CI results; TAX-40, TAX-51
NFR-09	Accessibility	Text and controls shall meet WCAG 2.1 AA contrast and size guidance so they stay readable on small screens and in sunlight.	Should	Design review of the design system (TAX-85)
NFR-10	Data use	Payment screens shall be lightweight to suit users with limited data.	Could	Page size check

Table 14: Non-functional requirements

6. DATA REQUIREMENTS

6.1 Logical Data

The data model has 12 tables, written in the schema file with a uuid primary key and real foreign keys, so the database rejects broken links. BetterAuth adds its own tables for users, sessions, accounts and verification. The entity relationship diagram is in Chapter 4.

Table	Holds	Links to
profiles	Name, phone, role (customer, driver, owner, association_admin, merchant_staff)	BetterAuth user
organisations	Associations and businesses; a business points to its association	Itself
memberships	Which profile belongs to which organisation, and in what role	profiles, organisations
vehicles	Registration, owner business, route	organisations, routes
stalls	Trader stalls and their owner business	organisations
routes	Route name, association and fare model (flat or ladder)	organisations
fareBands	Route, label, amount in cents, sort order	routes
trips	Vehicle, driver, route, start and end time	vehicles, profiles, routes
wallets	Owner type and owner, balance in cents	profiles or organisations
transactions	From and to wallet, amount, kind (top-up, fare, merchant), method (QR, NFC, simulated), quantity, trip, fare band, staff, location, idempotency key	wallets, trips, fareBands, profiles
qrTokens	Wallet, nonce, expiry time, time used	wallets
nfcTags	Tag serial, wallet, active flag	wallets

Table 15: Data tables

6.2 Data Rules

- Amounts are whole cents and never negative in a wallet.
- Each payment carries an idempotency key with a unique index.
- Transactions are recorded once and are not edited or deleted; a correction is a new transaction.
- Personal data is limited to name, contact detail and role (NFR-04). Test accounts and any research data are deleted after assessment unless the supervisor requires otherwise.

7. BUSINESS RULES

ID	Rule	Linked requirements
BR-01	The fare is the amount of the fare band chosen at boarding times the quantity of passengers, calculated on the server.	FR-06, FR-11, FR-21
BR-02	A fare moves money from the payer's wallet to the owner business's wallet; a merchant payment moves it to the merchant organisation's wallet. Money moves only through the transfer function.	FR-15, FR-26
BR-03	A payment is refused if it would take the payer's balance below zero.	FR-17
BR-04	A QR token is signed, names one wallet, expires after 60 seconds and can be used once.	FR-25
BR-05	A repeated payment with the same idempotency key returns the first result and charges once.	FR-17
BR-06	Only an association administrator can add or change routes and fare bands. A change applies to later payments only.	FR-06, FR-19
BR-07	A fare payment needs an open trip for the driver.	FR-20
BR-08	Top-ups are demo funds only and represent no real money.	FR-05
BR-09	A failed or incomplete payment is recorded as such and is never shown as successful.	FR-04

Table 16: Business rules

8. USE CASES

Table 17 lists the main use cases. The two most important are described in full below. The UML use case diagram is in Chapter 4.

ID	Use case	Actors	Requirements
UC-01	Register and sign in	All users	FR-01, FR-19
UC-02	Top up demo funds	Customer	FR-05, FR-02, FR-26
UC-03	Pay a fare by QR code at boarding	Customer, driver	FR-25, FR-07, FR-11, FR-10, FR-04, FR-17, FR-20, FR-26
UC-04	Pay by simulated NFC tap or tag	Customer, driver	FR-12, FR-24
UC-05	Pay a trader in merchant mode	Customer, trader	FR-09, FR-04
UC-06	View balance and transaction history	Customer	FR-02, FR-13
UC-07	Start and end a trip, and view the shift record	Driver	FR-20, FR-08
UC-08	View takings and the route map	Owner	FR-14, FR-22, FR-23
UC-09	Manage organisations, routes and fares	Association administrator	FR-15, FR-06

Table 17: Use case summary

UC-03: Pay a fare by QR code at boarding

Item	Description
Actors	Passenger (customer) and driver.
Preconditions	The driver is signed in with an open trip on a route. The passenger is signed in and has enough demo funds.

Item	Description
Main flow	1. The passenger opens the Pay screen, which shows a QR code with a signed token. 2. The driver chooses the fare band and the quantity on the reader screen. 3. The driver scans the QR code. 4. The server checks the token (signature, expiry, not yet used). 5. The server works out the fare from the fare band and quantity. 6. The transfer function debits the passenger, credits the owner business and records the transaction with the trip, driver, fare band and location (if allowed). 7. The reader shows success and the amount, and the passenger sees the new balance.
Alternative flows	A1. Token expired or already used: the payment is refused and the passenger refreshes the code. A2. Balance too low: the payment is refused and both screens show "failed". A3. Scan fails: the driver types the payment code (FR-16). A4. The same request arrives twice: the idempotency key returns the first result.
Postconditions	On success the balances and ledger agree and one transaction exists. On failure no balance changes.

Table 18: Use case UC-03

UC-05: Pay a trader in merchant mode

Item	Description
Actors	Trader (merchant_staff) and customer.
Preconditions	The trader is signed in and linked to a stall. The customer is signed in and has enough demo funds.
Main flow	1. The trader enters the amount and the screen shows a payment QR. 2. The customer scans the QR in the app. 3. The customer sees the merchant name and the amount, and confirms. 4. The server runs the transfer function from the customer's wallet to the merchant organisation's wallet. 5. Both screens show the outcome.
Alternative flows	A1. Balance too low: refused, both screens show "failed". A2. The customer cancels: nothing is charged.
Postconditions	One transaction of kind "merchant" is recorded on success.

Table 19: Use case UC-05

9. DESIGN-ONLY AND OUT-OF-SCOPE ITEMS

These are documented in the design but not built, in line with the topic acceptance conditions: reader hardware; payment cards and wristbands; a phone-free fallback for commuters; bank linking and PayShap top-ups (BusinessTech 2023); zero-rated or data-light access; an offline driver mode; live fare feeds from taxi associations; and any handling of real money, which would need a licensed payment partner and regulatory assessment.

10. REQUIREMENTS TRACEABILITY MATRIX

Table 20 traces each functional requirement to its source, its Linear issues and its test case. Test case IDs follow the pattern TC-<requirement ID> and are written in the test workbook (test plan and test cases, owned by Christoffel). The matrix is completed with test results in Week 4.

Requirement	Priority	Source	Linear issues	Test case
FR-01	Must	Stakeholders (3.2); NFR-03	TAX-31, TAX-32	TC-FR-01
FR-02	Must	US-01	TAX-36	TC-FR-02

Requirement	Priority	Source	Linear issues	Test case
FR-03	Must	Schalekamp et al (2017); Loop limits	TAX-36	TC-FR-03
FR-04	Must	Aruho et al (2021) (trust); testing focus	TAX-44, TAX-50	TC-FR-04
FR-05	Must	Simulated funds (Chapter 1); US-02	TAX-37	TC-FR-05
FR-06	Must	Research gap; US-04	TAX-47	TC-FR-06
FR-07	Must	Loop; WOW and Codeta hardware costs; US-05	TAX-43, TAX-48	TC-FR-07
FR-08	Must	BebaPay; Aruho et al (2021); US-06	TAX-52	TC-FR-08
FR-09	Should	Street Wallet; Table 2; US-08	TAX-55, TAX-56	TC-FR-09
FR-10	Must	Waxd; BebaPay; US-05	TAX-44, TAX-52	TC-FR-10
FR-11	Must	Research gap; US-04	TAX-44, TAX-49, TAX-51	TC-FR-11
FR-12	Should	Chapter 1 scope; US-03	TAX-46	TC-FR-12
FR-13	Must	US-10	TAX-39	TC-FR-13
FR-14	Should	Schalekamp et al (2017); Waxd; US-07	To be linked in Linear	TC-FR-14
FR-15	Must	Stakeholders (associations, owners)	TAX-34, TAX-35	TC-FR-15
FR-16	Should	Risk R04	TAX-43	TC-FR-16
FR-17	Must	Risk R11; NFR-05	TAX-38, TAX-44	TC-FR-17
FR-18	Should	Risk R09	TAX-29, TAX-93	TC-FR-18
FR-19	Must	Stakeholders (3.2); NFR-03	TAX-33	TC-FR-19
FR-20	Must	Boarding at ranks and roadside stops (2.1)	TAX-48, TAX-50, TAX-52	TC-FR-20
FR-21	Should	Scenario analysis	TAX-49	TC-FR-21
FR-22	Should	Owner accountability; scenario analysis	TAX-53	TC-FR-22
FR-23	Could	US-07	TAX-90	TC-FR-23
FR-24	Could	W3C Web NFC Community Group (n.d.); Chromium Projects (n.d.)	TAX-46, TAX-91	TC-FR-24
FR-25	Must	NFR-03; US-03	TAX-41, TAX-42	TC-FR-25
FR-26	Must	Risk R11; NFR-05	TAX-38, TAX-40	TC-FR-26

Table 20: Requirements traceability matrix

11. OPEN ISSUES

#	Issue	Owner	Needed by
1	The data model draft has no status field on transactions, but FR-04 needs successful, failed and pending outcomes. Add a status field or store failed attempts separately.	Christoffel (ERD)	Before schema merge
2	Choose the QR scanner and generator library; it must scan on Android Chrome and iOS Safari.	Christoffel	17 October
3	Confirm hosting on Vercel and Neon.	Group, at stand-up	Week 4
4	Google Maps needs a Google Cloud billing account; restrict the key and watch usage.	Lethukuthula	Week 4
5	Link FR-14 (owner takings) to its Linear issues	Lethukuthula	Week 4
6	Confirming ethics approval before any interviews, surveys or usability tests with people	Group	Before Sprint 2 usability test
7	Replace the dummy fare table with real association fares if access is granted (risk R02)	Group	If access is granted
8	Confirm the Week 3 repository link on the cover page	Christoffel	16 October

Table 21: Open issues

CHAPTER 4: SYSTEM DESIGN

4.2 System architecture and technology stack

Taxi Tap is designed as a single, mobile-first Next.js web application. Passengers, drivers, merchants, taxi owners and association administrators access role-specific screens through a phone or desktop browser. The interface communicates with application logic through Next.js server actions; the browser does not access the database directly.

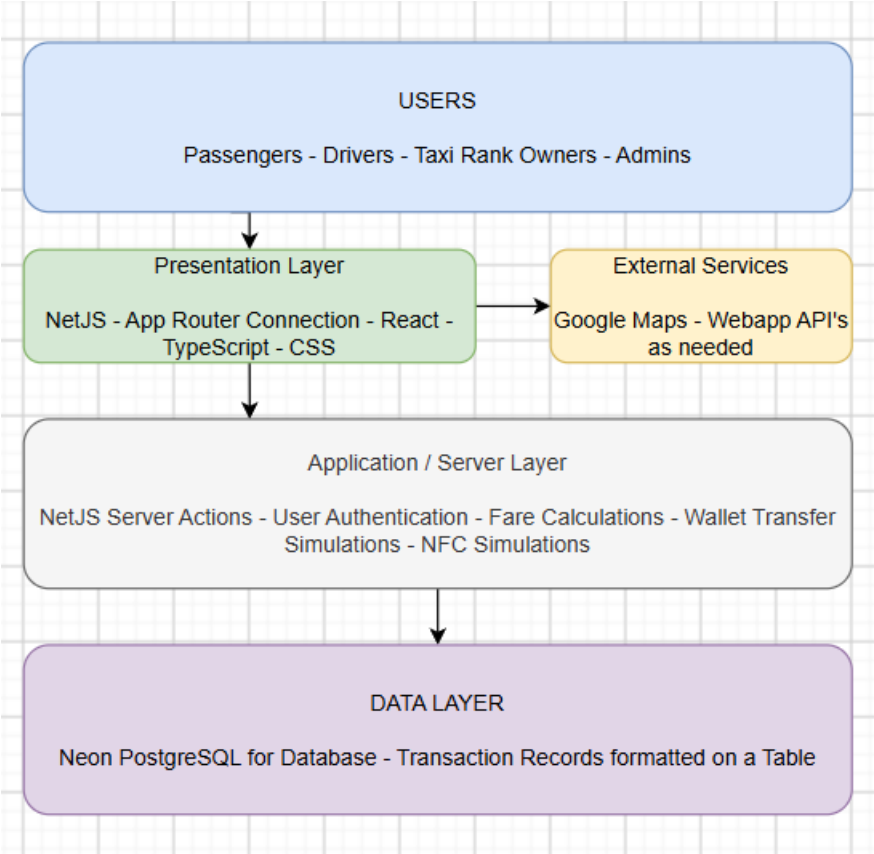


Figure 1: Proposed system architecture for Taxi Tap

The server layer checks the signed-in user's role before processing each action. It validates QR tokens, calculates fares from the selected fare band and passenger quantity, checks the payer's balance and processes wallet transfers.

Using Neon PostgreSQL or our chosen database language if we change, is where wallet and ledger changes are written atomically.

BetterAuth manages authentication and sessions in the same database. Google Maps JavaScript API is used for route and payment-location views.

Optional Web NFC reading is limited to compatible Android Chrome devices over HTTPS; simulated NFC remains the fallback.
All funds in the prototype are simulated, and the system does not connect to banks or payment providers.

Table 1: Technology stack and justification

Layer / purpose	Selected technology	Reason for selection	Alternative / limitation
Web application	Next.js App Router, React, TypeScript	Single application for role-based screens and server actions; TypeScript supports maintainable code.	Separate frontend and API would add deployment and integration work.
UI styling	shadcn/ui and Tailwind CSS	Reusable components and responsive, mobile-first styling.	Custom components would take longer to implement consistently.
Server logic	Next.js Server Actions	Keeps sensitive payment, fare and role checks on the server.	Client-side-only processing is not appropriate for payment rules.
Database	Neon PostgreSQL	Relational data, foreign keys and database transactions support consistent wallet and ledger records.	A non-relational database is less aligned with the relational entities and constraints.
ORM / access	Drizzle ORM with Neon serverless driver	Typed schema and migrations provide controlled access to PostgreSQL; transactions support atomic transfers.	Direct SQL in screens would weaken separation of concerns.
Authentication	BetterAuth	Handles sign-in and sessions and stores authentication data in PostgreSQL.	A custom authentication implementation increases security and maintenance risk.
Route map	Google Maps JavaScript API	Supports owner route maps and payment-location pins.	Map features are limited if the external API key or billing is not configured.
Hosting	Vercel (proposed)	Fits the Next.js application deployment model.	Hosting choice remains to be confirmed by the group.
QR / NFC	QR generation and scanning library; optional Web NFC	QR is the main payment identifier; Web NFC can be used on compatible Android browsers.	The SRS notes that the QR library choice must be confirmed; Web NFC is not universal.
Version control and tests	GitHub, CI, Vitest	Pull requests, linting, type-checking and unit tests support team collaboration and repeatable checks.	Manual checks alone provide weaker regression protection.

The stack reflects the SRS and the Group 4 technology-stack document. Vercel hosting and the QR library are recorded as decisions requiring confirmation rather than treated as already finalised.

4.3 UML models

Use case diagram

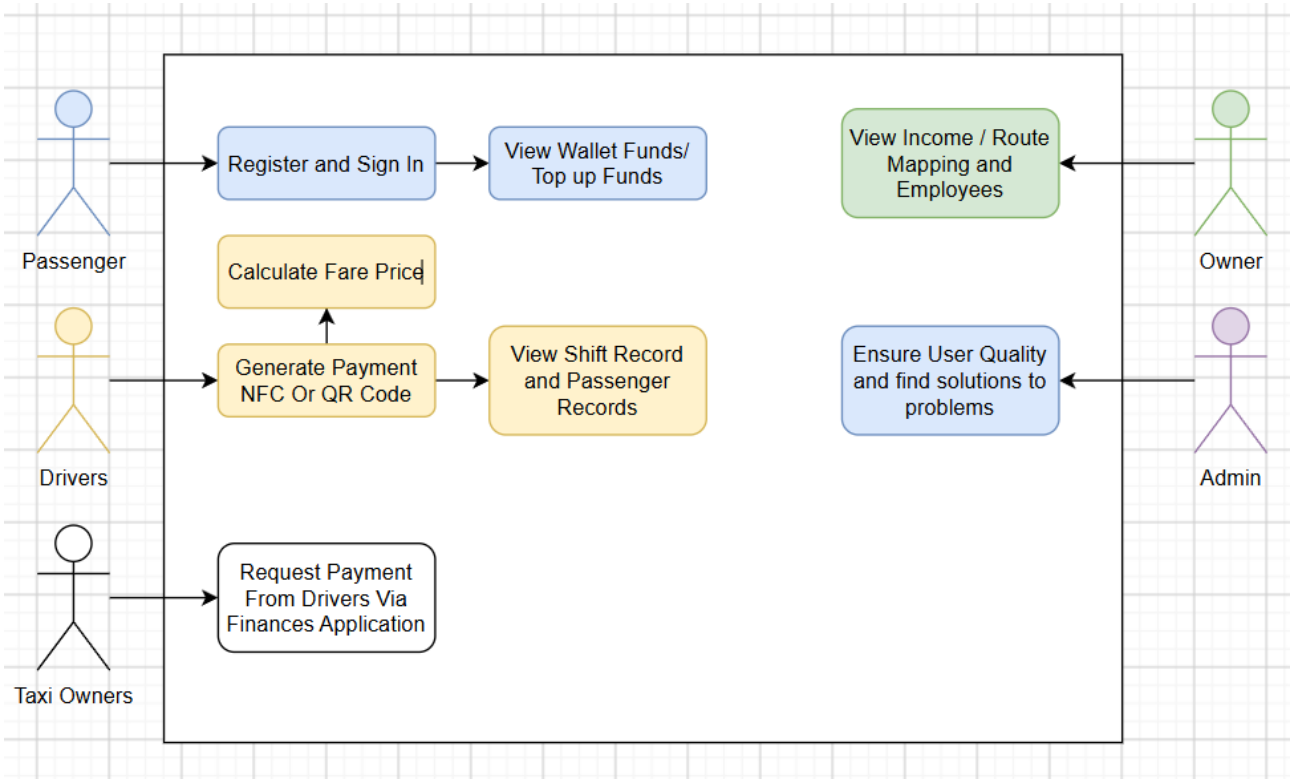


Figure 2: Use case overview for the Taxi Tap system

The diagram groups functions by the five roles in the SRS. Passengers manage their own wallet, display payment QR codes and review transaction history.

Drivers start trips, select fare bands, scan passenger QR codes and view shift records. Merchants request payments. Owners review takings and map information.

Association administrators manage organisations, memberships, routes and fare bands. Authentication and server-side role checks apply before protected functions are executed.

Sequence diagram

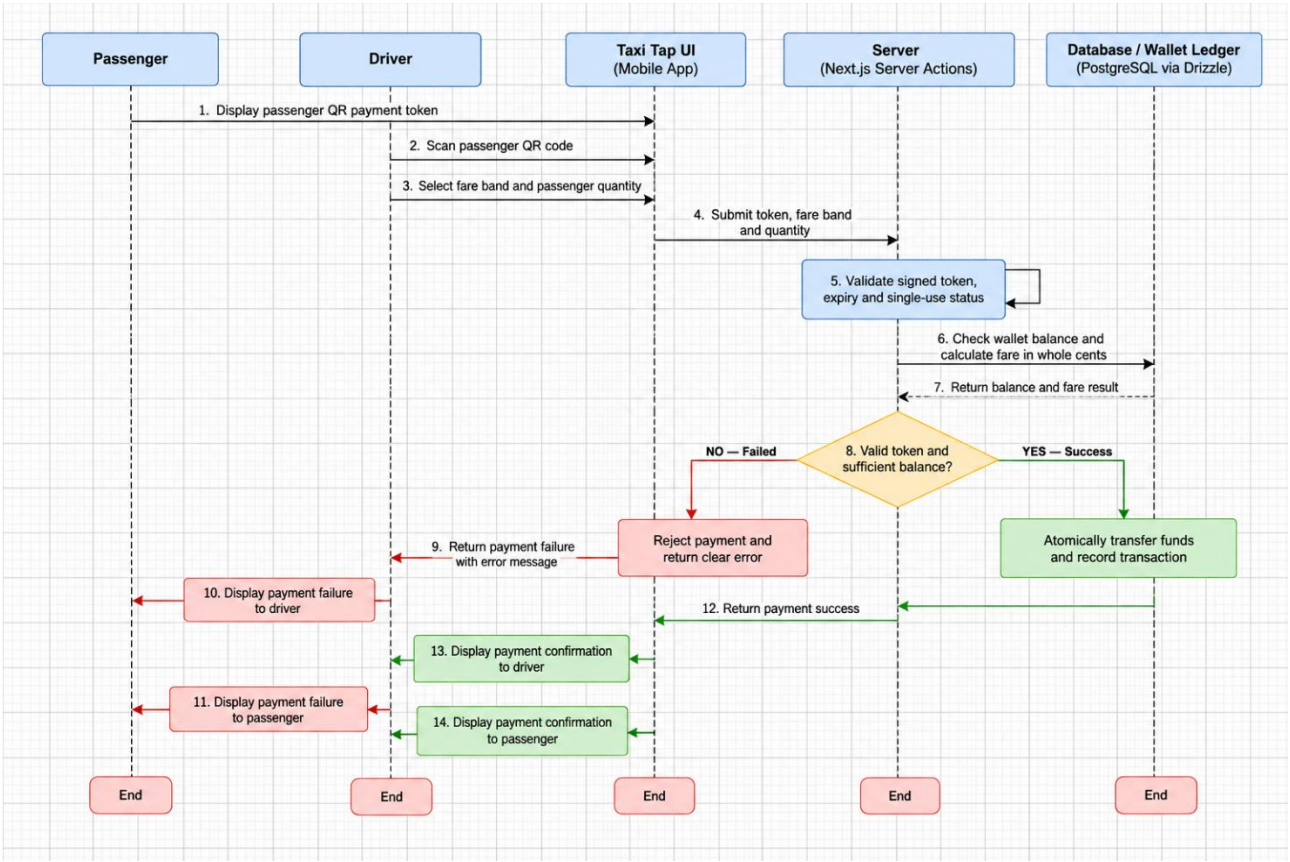


Figure 3: Sequence diagram for a QR fare payment

The driver submits the scanned token, selected fare band and passenger quantity to a server action. The server verifies the token signature, expiry and single-use status, calculates the fare and checks the idempotency key and wallet balance.

If valid, a database transaction debits the payer, credits the owner business and records the ledger transaction together.

If any step fails, the transfer must not leave a partial balance change. The server returns a success or failure result for display. A repeated idempotency key returns the original result and must not create a second charge.

Activity diagram

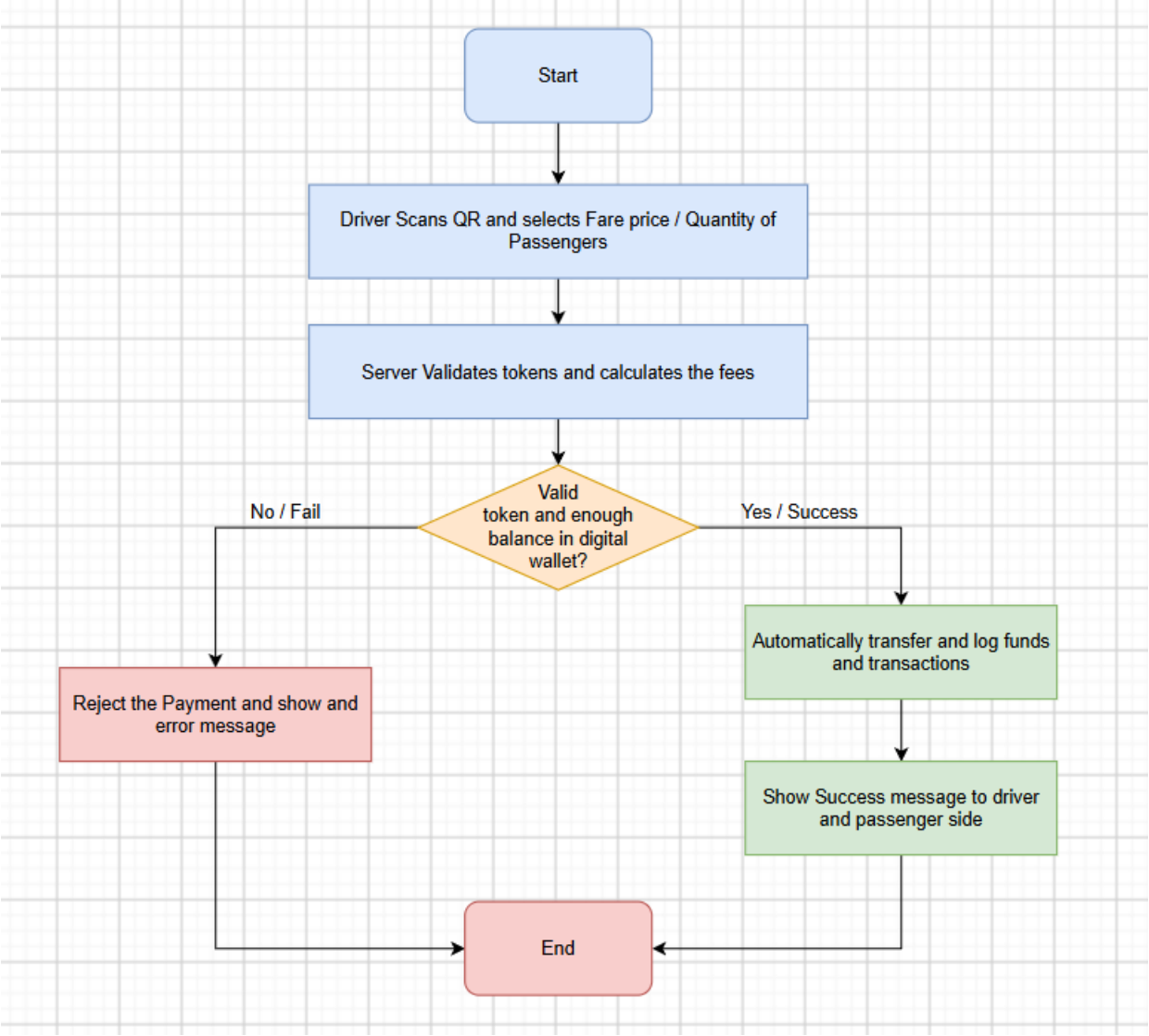


Figure 4: Activity diagram for the fare payment decision

The activity diagram emphasises the validation decision. A token that is invalid, expired or already used, or a payment that would reduce the payer's balance below zero, is rejected with a clear message.

A valid payment is transferred and recorded atomically before confirmation is shown.

This supports the SRS requirements for payment confirmation, low-balance refusal and prevention of duplicate charges.

4.4 Database design

The logical database design follows the 12 application tables listed in the SRS. UUID primary keys and foreign keys are used to identify records and prevent broken relationships. BetterAuth adds its own authentication tables for users, sessions, accounts and verification. The following ERD is a logical design overview; exact column names and relationship cardinalities must be checked against the team's Drizzle schema before the final submission.

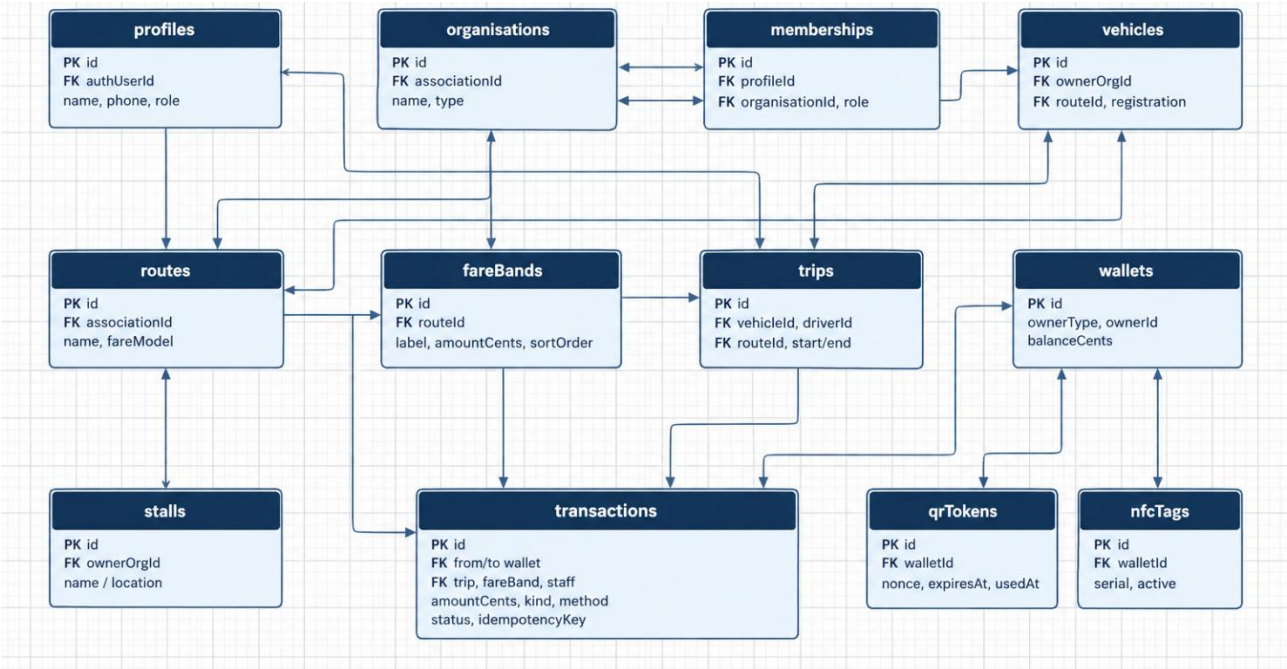


Figure 5: Logical entity relationship overview for Taxi Tap

The core relationships include routes with multiple fare bands and also vehicles associated with owner organisations and routes.

The trips link to a vehicle, driver and route, then wallets owned by profiles or organisations as well as and transactions linking payer and recipient wallets with the relevant trip, fare band and staff member.

QR tokens and NFC tags identify a wallet. Memberships users like the passengers then connect profiles to organisations and record their role.

Table 2: Application data dictionary summary

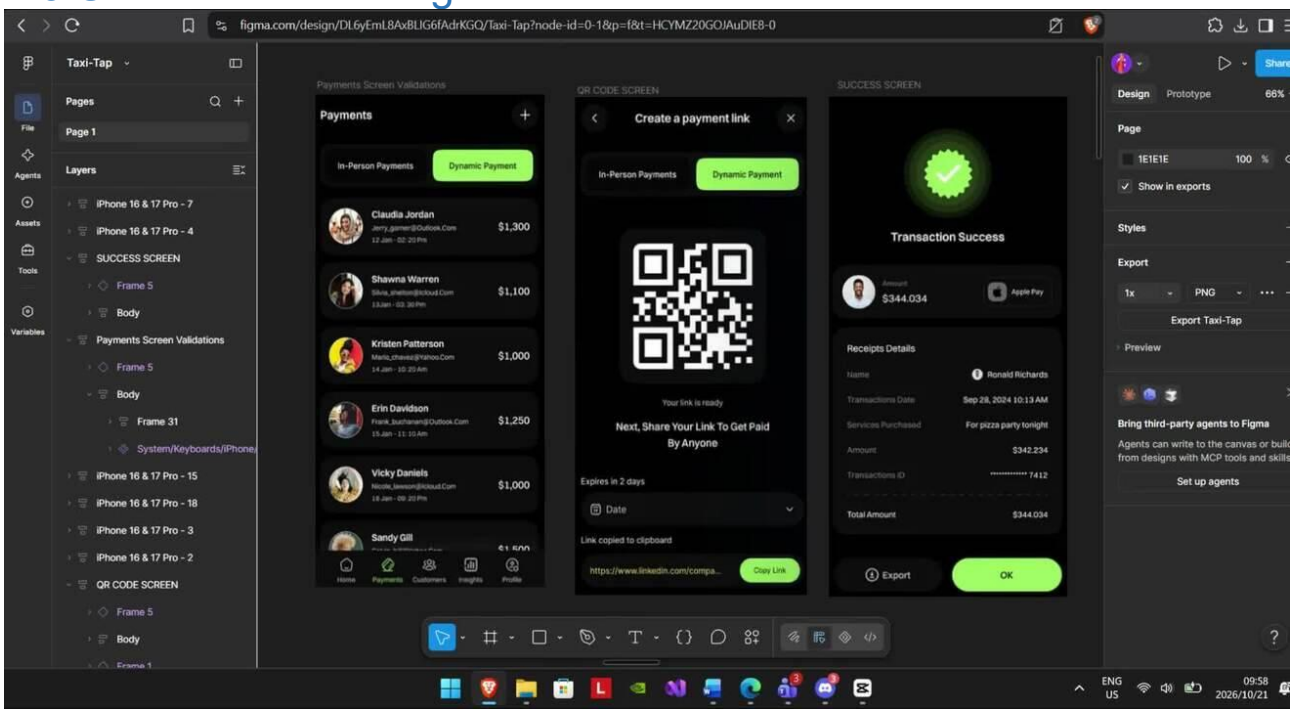
Table / entity	Purpose	Key links and design notes
profiles	Application profile, contact details and role.	Linked to BetterAuth user; roles include customer, driver, owner, association_admin and merchant_staff.
organisations	Associations and businesses.	An organisation may point to its parent association.
memberships	Connects profiles to organisations and their roles.	Foreign keys to profiles and organisations.
vehicles	Taxi vehicles and registration details.	Linked to owner business and route.
stalls	Trader stalls and ownership.	Linked to the owner business / organisation.
routes	Route name, association and fare model.	Linked to association; fare model may be flat or ladder.
fareBands	Fare options for a route.	Linked to route; amount stored in whole cents and sorted for display.
trips	A driver's operating period for a vehicle on a route.	Linked to vehicle, driver profile and route; payments reference the open trip.
wallets	Wallet owner and balance.	Owned by a profile or organisation; balance must never be negative.
transactions	Immutable record of money movement.	From/to wallet, amount in cents, kind, method, quantity, trip, fare band, staff, location and unique idempotency key.
qrTokens	Short-lived QR payment tokens.	Linked to wallet; signed, expires after 60 seconds and can be used once.
nfcTags	Registered NFC tag details.	Linked to wallet; stores serial and active status.

Data integrity and design decisions

- Money amounts are stored as whole cents (decibles) almost like floats, avoiding bad rounding for fare and wallet calculations.
- Wallet balances cannot be negative. A transfer must debit and credit wallets and write the corresponding transaction within one database transaction.
- Every payment uses a unique idempotency key so retries do not charge the passenger twice.
- Transactions are treated as an audit ledger: they are not edited or deleted; corrections are recorded as new transactions.
- Fare amounts and passenger quantity are combined on the server, rather than trusting a total submitted by the browser.
- QR tokens are signed, expire after 60 seconds and are single use codes and the server validates the token.
- Personal information is limited to the name, contact details and role needed by the prototype. Fictional demo data is used.

The SRS open-issues list identifies that transaction status needs to represent successful, failed and pending outcomes. The schema should include a status field, or store failed attempts separately, before the design is baselined. The ERD and data dictionary must be updated to match the final implementation.

4.5 User interface design



Taxi Tap uses a mobile-first interface because passengers, drivers and merchants are expected to use phones while travelling or working. Owner and association administration screens may also be used on desktop browsers. The wireframes below show the intended screen structure rather than a screenshot of a completed application. The design uses the purple visual style described in the SRS, large touch targets, readable contrast and clear success or failure feedback.

Figure 6: Initial wireframes for passenger, driver and owner screens

Table 3: User interface design decisions

Screen / area	Design decision	Reason
Passenger wallet	Balance is prominent; QR display and demo top-up are clear actions; recent transactions appear below.	Reduces steps to payment and helps passengers check their balance and payment history.
Driver reader	Active route and fare band appear before scanning; passenger quantity and scan action are easy to reach.	Supports fast boarding and reduces fare selection mistakes.
Payment result	Show an immediate, unambiguous success or failure message.	Lets the driver confirm payment without counting cash or giving change.
Merchant mode	Merchant enters an amount and displays a payment QR.	Allows a trader to accept wallet payments without a card machine.
Owner dashboard	Show takings by taxi and driver, with a route map where available.	Supports reconciliation and gives owners a clearer record of takings.
Association administration	Group organisation, membership, route and fare management functions together.	Keeps administrative tasks distinct from passenger and driver tasks.
All screens	Responsive layout, high-contrast text, large touch targets and consistent navigation.	Supports small screens, outdoor use and the SRS usability/accessibility requirements.

The initial wireframes cover passenger, driver and owner views.

Merchant and association administrator screens must also be wireframed in the next design iteration so all five user roles in the SRS are represented.

The final wireframes should be checked against the implemented screens and the SRS acceptance criteria.

WEEKLY PROGRESS REPORT AND UPDATED REGISTERS

Weekly Progress Report

Field	Report
Reporting period	Week 3 planning window: 12–20 October 2026
Prepared for	Ms Amenda Raganya, Project Supervisor
Project	Taxi Tap: Digital Wallet for Minibus Taxi Fares
Overall status	Documentation baseline assembled; implementation and test evidence must be verified against the live repository.
Week 3 target	Chapter 3, SRS, Chapter 4 sections 4.2–4.5, revised Chapters 1–2, schema and first commits from all members.
Next milestone	Week 4 build and test due 23 October 2026.
Final milestone	Final dissertation, source code, evidence folders, presentation and live demo due 29 October 2026.

Progress by team member

Member	Area / deliverables	Next planned work	Blocker / note
Lethukuthula Gumede	Group lead, UI/UX, DevOps, backend support, project plan/registers and Chapter 4.	Continue app integration, auth/organisations/wallet/ledger support; update registers and deploy over HTTPS. Host payments	Group leadr is covering documentation while Londa is unavailable, according to the Week 2 plans
Londa Mavimbela	Backend owner for payment flow, QR tokens, fare engine, merchant payments and seed data.	Implement/review wallet transfers, QR token expiry and fare validation; review backend pull requests	Role remains backend owner even while documentation coverage is reassigned
Christoffel Du Toit	Frontend, QR flow, ERD/schema, QA, test workbooks and final compilation.	Complete passenger/merchant/boarding reader screens, test cases and final document integration	Repository commits and executed tests need evidence from the live tools

Updated work plan

ID	Task	Owner	Start	Due	Status / note
T1–T3	Kick-off, proposal, repository and project setup	Group	28 Sep	8 Oct	Completed according to Week 2 plan
T4	Chapter 1: Introduction and Problem Definition	Christoffel	5 Oct	8 Oct	Draft carried forward; alignment checked
T5	Literature search and Chapter 2	Group	5 Oct	9 Oct	Draft carried forward; alignment checked
T6–T7	Project plan, ethics and risk planning	Lethukuthula / Group	5 Oct	9 Oct	Plan baseline exists; update included here
T8–T11	Requirements gathering, Chapter 3, SRS and design	Group	12 Oct	16 Oct	Documents assembled; check against supervisor feedback
T12	Database schema and migrations	Londa / Lethukuthula / Christoffel	14 Oct	23 Oct	Implementation status must be confirmed from repository
T13	Wallet, ledger and payment flow	Londa with support	16 Oct	23 Oct	In progress / verify from repository
T14	Passenger, merchant and driver screens	Christoffel	16 Oct	23 Oct	In progress / verify from repository

T15	Role checks, QR expiry, idempotency and atomicity tests	Group	16 Oct	23 Oct	Planned Must-feature testing
T16	Walkthrough, test workbooks and Chapter 5	Group	20 Oct	23 Oct	Upcoming
T17	Final dissertation and demo preparation	Group	23 Oct	29 Oct	Upcoming

Milestone status

Milestone	Due date	Deliverables	Status
Week 1 – Proposal	2 October 2026	Proposal and topic acceptance	Completed according to the project plan
Week 2 – Problem and literature	9 October 2026	Chapters 1–2, project plan and repository	Completed as a draft submission
Week 3 – Requirements and design	16 October 2026	Chapter 3, SRS, Chapter 4.2–4.5, revised Ch1–2, schema and commits	Documentation assembled; repository evidence must be checked
Week 4 – Build and test	23 October 2026	Working Must features, walkthrough, Chapters 4.6–4.8, Chapter 5 and test workbooks	Upcoming
Final submission	29 October 2026	Full dissertation, source code, evidence, presentation and live demo	Upcoming

Risk, Issue and Change Register

ID	Type	Risk / issue / change	Impact	Mitigation / action	Owner	Status
R01	Risk	Four-week deadline may leave insufficient time for all optional features.	Must features or testing could be incomplete.	Prioritise Must requirements; move Should/Could features behind core payment, fare and ledger flows.	Group	Open / monitor
R02	Risk	Taxi association access and live fare data may be delayed.	Real-world validation and fare accuracy may be limited.	Use clearly labelled dummy routes/fare bands for prototype tests; do not present dummy fares as official fares.	Group lead	Open / monitor
R03	Risk	Ethics approval or informed consent may not be complete in time.	Human interviews, surveys or observations cannot be conducted.	Use document review and scenario analysis; collect no research data from people until approval and consent are confirmed.	Group	Open / monitor
R04	Risk	Web NFC is only supported on compatible Android Chrome devices over HTTPS.	NFC demonstration may fail on some phones.	Treat NFC as optional; provide simulated NFC fallback and make QR the main payment path.	Londa / Christoffel	Open / monitor
R05	Risk	Wallet retries or concurrent requests could cause duplicate charges or incorrect balances.	Financial integrity and trust could be affected.	Use a single database transaction, unique idempotency keys, server-side validation and reconciliation tests.	Backend owner	Open / monitor
R06	Risk	Secrets or personal information could	Security and privacy exposure.	Use fictional demo data, environment files, secret settings and repository	Group lead	Open / monitor

		be committed to version control.		checks; do not commit keys.		
I01	Issue	Live GitHub repository URL and commit screenshots are absent from the source documents supplied for this compilation.	Development evidence cannot be independently verified in this document.	Insert the actual repository URL and export/screenshots of commits from each member.	Group lead	Action required
I02	Issue	Transaction outcomes need to represent successful, failed and pending states.	The transaction ledger may not explain every payment attempt clearly.	Add a transaction status field or separately persist failed attempts before schema baseline; align ERD and tests.	Backend / schema owners	Action required
C01	Change	NFC is optional and simulated funds remain the only funds used in the prototype.	Defines feasible scope and avoids implying real payment processing.	Keep bank linking, physical reader hardware and zero-rated data outside implementation scope.	Group	Accepted scope
C02	Change	Fare totals are calculated on the server using the selected fare band and passenger quantity.	Prevents browser-submitted amounts being trusted.	Keep the server calculation as the source of truth and test fare boundaries.	Backend owner	Accepted design

Submission completion check

Deliverable	Included in this document	Final verification needed
Chapter 3: Requirements and Methodology	Yes	Remove any remaining drafting notes after group review
Software Requirements Specification	Yes	Confirm requirement IDs against actual Linear issues and tests
Chapter 4: System Design, sections 4.2–4.5	Yes	Validate diagrams and ERD against final schema
Development evidence	Structure and evidence checklist included	Add actual GitHub URL, schema script and commit evidence
Revised Chapters 1 and 2	Existing chapter content and alignment record included	Confirm any supervisor-specific feedback and update text accordingly
Weekly progress report and updated registers	Included	Confirm live task status with the team

. REFERENCES

- Aruho, A.T., Behrens, R., Mitullah, W. and Kamau, A., 2021. *A case study of matatu cashless fare collection initiatives in Nairobi*. Paper presented at the 39th Southern African Transport Conference, 5–7 July 2021. Available at: https://repository.up.ac.za/handle/2263/82383 (Accessed: 9 October 2026).
- Beck, K., Beedle, M., van Bennekum, A. *et al*., 2001. *Manifesto for Agile Software Development*. Available at: https://agilemanifesto.org/ (Accessed: 16 October 2026).
- Better Auth, n.d. *Drizzle adapter*. Available at: https://www.better-auth.com/docs/adapters/drizzle (Accessed: 15 October 2026).
- BusinessTech, 2023. *New payment system launches in South Africa – with no need for cash*. Available at: https://businesstech.co.za/news/banking/672009/reserve-bank-launches-new-payment-system-in-south-africa-with-no-need-for-cash/ (Accessed: 9 October 2026).
- Chromium Projects, n.d. *Web NFC*. Available at: https://www.chromium.org/developers/design-documents/web-nfc/ (Accessed: 9 October 2026).
- Drizzle, n.d. *Connect Neon*. Available at: https://orm.drizzle.team/docs/connect-neon (Accessed: 15 October 2026).
- Group 4, 2026a. *Tech Stack and Project Structure*. Internal team document, 15 October 2026.
- Group 4, 2026b. *Chapter 3: Requirements and Methodology (draft)*. Weekly Submission 3, 16 October 2026.
- Joseph, N., 2015. *Matatus 1, Google 0: A sobering lesson on digital payments from Africa*. PaymentsJournal, 9 April. Available at: https://www.paymentsjournal.com/matatus-1-google-0a-sobering-lesson-on-digital-payments-from-africa/ (Accessed: 9 October 2026).
- Mondato, n.d. *Cashless transportation: Kenya and Rwanda as case studies*. Available at: https://blog.mondato.com/cashless-transportation-kenya-rwanda/ (Accessed: 9 October 2026).
- Neon, n.d. *Branching*. Available at: https://neon.com/docs/introduction/branching (Accessed: 15 October 2026).
- Republic of South Africa, 2013. *Protection of Personal Information Act 4 of 2013*. Pretoria: Government Printer.
- Royce, W.W., 1970. Managing the development of large software systems. *Proceedings of IEEE WESCON*, 26, pp. 1–9.
- Schalekamp, H., McLaren, M. and Behrens, R., 2017. *Exploring cashless fare collection in the context of urban public transport reform in South Africa*. Paper presented at the 36th Southern African Transport Conference, Pretoria, 10–13 July 2017. Available at: https://repository.up.ac.za/handle/2263/62743 (Accessed: 9 October 2026).
- Schwaber, K. and Sutherland, J., 2020. *The Scrum Guide*. Available at: https://scrumguides.org/ (Accessed: 16 October 2026).

- TechCabal, 2025. *South Africa's Street Wallet raises $350,000 to expand QR payments for informal traders*, 13 August. Available at: https://techcabal.com/2025/08/13/street-wallet-raises-350k/ (Accessed: 9 October 2026).
- The Standard, 2015. *Matatu operators opposed to BebaPay*. Available at: https://www.standardmedia.co.ke/business/article/2000105373/matatu-operators-opposed-bebapay (Accessed: 9 October 2026).
- Tinka, A.A. and Behrens, R., 2019. *Cashless fare collection in Sub-Saharan African paratransit: A review of experiences*. Paper presented at the 38th Southern African Transport Conference, Pretoria, 8–11 July 2019. Available at: https://repository.up.ac.za/handle/2263/74251 (Accessed: 9 October 2026).
- W3C Web NFC Community Group, n.d. *Web NFC*. Available at: https://w3c.github.io/web-nfc/ (Accessed: 9 October 2026).

Gant Chart

